

BOARD GAME CAFE MANAGEMENT SYSTEM

BY **JIYA ARORA 24BAI10034** INSTRUCTED BY @Sir Jayanth Velligiri

1. Title

Board Game Café Management System using FastAPI and MongoDB

2. Problem Statement

Many board game cafes manage their games, tables, and customer sessions manually using notebooks, spreadsheets, or basic records. This approach can lead to several operational challenges such as:

- Difficulty tracking which games are currently being used.
- Lack of visibility into table availability.
- Manual errors in managing customer sessions.
- No centralized system to maintain game and category information.
- Difficulty generating reports about popular games and table usage.
- Limited ability to analyze customer activity and cafe performance.

As the number of customers and games increases, manual management becomes inefficient and time-consuming.

To address these challenges, a centralized Board Game Cafe Management System was developed using FastAPI and MongoDB. The system automates game management, table allocation, session tracking, and report generation through REST APIs. It provides real-time information about game availability, table occupancy, session history, and operational analytics, thereby improving efficiency and reducing manual effort.

3. Objective

The objective of this project is to develop a backend-based management system for a board game cafe using FastAPI and MongoDB. The system manages game categories, board games, café tables, and customer gaming sessions through REST APIs. It also provides reporting and analytics features to monitor cafe operations and game usage.

4. Project Overview

The Board Game Cafe Management System is designed to automate the management of a modern board game cafe. Instead of maintaining records manually, the system allows cafe staff to manage games, allocate tables, track active sessions, and generate reports through API endpoints.

The project is developed using **FastAPI** as the backend framework and **MongoDB** as the database. **Swagger UI** is used for API testing and documentation, while **MongoDB Compass** is used to visualize and manage the stored data.

5. Outcome

After successful implementation, the system is capable of:

- Managing board game categories.
 - Managing board games available in the cafe.
 - Managing seating tables and their availability.
 - Starting and ending customer gaming sessions.
 - Automatically updating table and game status.
 - Tracking active and completed sessions.
 - Generating reports and analytics.
 - Identifying the most played games.
 - Identifying the most frequently used tables.
 - Providing a dashboard summary of cafe activities.
-

6. Tools and Technologies Used

Tool	Purpose
Python	Programming Language
FastAPI	Backend Framework
Uvicorn	ASGI Server
MongoDB Community Server	Database
MongoDB Compass	Database Visualization
PyMongo	MongoDB Driver
Pydantic	Data Validation
Swagger UI	API Testing & Documentation
Visual Studio Code	Development Environment

7. Database Design

The project consists of four primary collections:

1. Categories Collection

Stores game categories.

Example:

```
{  
  "category_id": "CAT001",  
  "name": "Strategy",  
  "description": "Strategic planning and decision making games"  
}
```

2. Games Collection

Stores all board games available in the café.

Example:

```
{
  "game_id": "GAME001",
  "name": "Catan",
  "category_id": "CAT001",
  "status": "Available"
}
```

3. Tables Collection

Stores café seating tables.

Example:

```
{
  "table_id": "TABLE001",
  "name": "Dragon Table",
  "capacity": 4,
  "status": "Available"
}
```

4. Sessions Collection

Stores customer gaming sessions.

Example:

```
{
  "session_id": "SESSION001",
  "group_name": "Weekend Gamers",
  "table_id": "TABLE001",
  "game_ids": ["GAME001", "GAME002"],
  "player_count": 4,
  "status": "Active"
}
```

8. Relationships Implemented

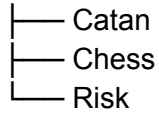
One-to-Many Relationship

Category -> Games

One category can contain multiple board games.

Example:

Strategy



Many-to-One Relationship

Games -> Category

Many games belong to one category.

One-to-Many Relationship

Table -> Sessions

One table can host multiple sessions over time.

Many-to-Many Relationship

Sessions <-> Games

One session can include multiple games and a game can participate in multiple sessions over time.

9. Features Implemented

Category Management

- Create Category
- View Category
- Update Category
- Delete Category

Game Management

- Create Game
- View Game
- Update Game
- Delete Game

Table Management

- Create Table
- View Table
- Update Table
- Delete Table

Session Management

- Start Session
- End Session
- Add Game to Existing Session
- View Session Details
- View Active Sessions

Reporting & Analytics

- Dashboard Summary
 - Session History Summary
 - Most Played Games Report
 - Most Used Tables Report
-

10. Additional Functionalities Beyond CRUD

This project includes several functionalities beyond standard CRUD operations.

Session Lifecycle Management

When a session starts:

- Table status changes from Available to Occupied.
- Game status changes from Available to Being Played.

When a session ends:

- Table status changes back to Available.
- Game status changes back to Available.

Status Tracking

The system continuously tracks:

- Active sessions

- Completed sessions
- Available games
- Games currently in use
- Available tables
- Occupied tables

Analytics and Reporting

MongoDB Aggregation Pipelines are used to generate analytical reports such as:

- Most played games.
- Most frequently used tables.
- Session statistics.
- Dashboard overview.

These features make the project more advanced than a basic CRUD application.

11. API Endpoints

Categories

GET /categories/

POST /categories/

GET /categories/{category_id}

PUT /categories/{category_id}

DELETE /categories/{category_id}

Games

GET /games/

POST /games/

GET /games/{game_id}

PUT /games/{game_id}

DELETE /games/{game_id}

Tables

GET /tables/

POST /tables/

GET /tables/{table_id}

PUT /tables/{table_id}

DELETE /tables/{table_id}

Sessions

GET /sessions/

GET /sessions/{session_id}

DELETE /sessions/{session_id}

POST /sessions/start

POST /sessions/end/{session_id}

POST /sessions/{session_id}/add-game

GET /sessions/details/{session_id}

Reports

GET /reports/dashboard

GET /reports/session-history

GET /reports/most-played-games

GET /reports/most-used-tables

12. Output Screenshots

Screenshot 1 – Swagger Home Page

Board Game Café Management System 1.0.0 QA 1.5.1

/api/swagger.json

A MongoDB + FastAPI project to manage a board game café: categories, games, tables, and play sessions.

Categories

GET	/categories/	Get All Categories	⌵
POST	/categories/	Create Category	⌵
GET	/categories/{category_id}	Get Category	⌵
PUT	/categories/{category_id}	Update Category	⌵
DELETE	/categories/{category_id}	Delete Category	⌵

Games

GET	/games/	Get All Games	⌵
POST	/games/	Create Game	⌵
GET	/games/{game_id}	Get Game	⌵
PUT	/games/{game_id}	Update Game	⌵
DELETE	/games/{game_id}	Delete Game	⌵

Tables

GET	/tables/	Get All Tables	⌵
POST	/tables/	Create Table	⌵
GET	/tables/{table_id}	Get Table	⌵
PUT	/tables/{table_id}	Update Table	⌵
DELETE	/tables/{table_id}	Delete Table	⌵

Sessions

GET	/sessions/	Get All Sessions	⌵
GET	/sessions/{session_id}	Get Session	⌵
DELETE	/sessions/{session_id}	Delete Session	⌵
POST	/sessions/start	Start Session	⌵
POST	/sessions/end/{session_id}	End Session	⌵
POST	/sessions/{session_id}/add-game	Add Game To Session	⌵
GET	/sessions/details/{session_id}	Get Session Details	⌵

Root

GET	/	Root	⌵
-----	---	------	---

Reports

GET	/reports/dashboard	Dashboard Report	⌵
GET	/reports/session-history	Session History	⌵
GET	/reports/most-played-games	Most Played Games	⌵
GET	/reports/most-used-tables	Most Used Tables	⌵

Description:

Displays all available API endpoints grouped into Categories, Games, Tables, Sessions, and Reports.

Screenshot 2 – Category Creation

Categories

GET /categories/ Get All Categories

POST /categories/ Create Category

Create a new category.
Steps:
1. Check that category_id is not already used.
2. Insert the new category document into MongoDB.

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value | Schema

```
{  "category_id": "CAT001",  "name": "Strategy",  "description": "Strategic planning and decision making games"}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  'http://127.0.0.1:8080/categories/' \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "category_id": "CAT001",  "name": "Strategy",  "description": "Strategic planning and decision making games"  }'
```

Request URL

http://127.0.0.1:8080/categories/

Server response

Code | Details

200

Response body

```
{  "category_id": "CAT001",  "name": "Strategy",  "description": "Strategic planning and decision making games"}
```

Download

Response headers

```
content-length: 183  content-type: application/json  date: Thu, 18 Jun 2026 13:37:35 GMT  server: unicorn
```

Responses

Code | Description | Links

200 | Successful Response | [No links](#)

Screenshot 3 – Game Creation

Games

GET /games/ Get All Games

POST /games/ Create Game

Create a new game.
Steps:

1. Check that game_id is not already used.
2. Check that the referenced category_id actually exists (this enforces the Games -> Category relationship).
3. Insert the new game document.

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value | Schema

```
{  "game_id": "GAME010",  "name": "Cards Against Humanity",  "category_id": "CAT002",  "min_players": 4,  "max_players": 12,  "status": "Available"}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  http://127.0.0.1:8000/games/ \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "game_id": "GAME010",  "name": "Cards Against Humanity",  "category_id": "CAT002",  "min_players": 4,  "max_players": 12,  "status": "Available"  }'
```

Request URL

http://127.0.0.1:8000/games/

Server response

CodeDetails

200

Response body

```
{  "game_id": "GAME010",  "name": "Cards Against Humanity"
```

Description:

Demonstrates insertion of a board game linked to a category.

Screenshot 4 – Table Creation

Tables

GET /tables/ Get All Tables

POST /tables/ Create Table

Create a new table. table_id must be unique.

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "table_id": "TABLE005",
  "name": "Castle Table",
  "capacity": 10,
  "status": "Available"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/tables/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "table_id": "TABLE005",
    "name": "Castle Table",
    "capacity": 10,
    "status": "Available"
  }'
```

Request URL

http://127.0.0.1:8000/tables/

Description:
Demonstrates insertion of a café seating table.

Screenshot 5 – Session Start

Sessions

GET /sessions/ Get All Sessions

GET /sessions/{session_id} Get Session

DELETE /sessions/{session_id} Delete Session

POST /sessions/start Start Session

Start a new session.

- Validates table and games.
- Creates the session.
- Marks table as Occupied.
- Marks games as Being Played.

See services/session_service.py for the full logic.

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{  "session_id": "SESSION002",  "group_name": "College Legends",  "table_id": "TABLE001",  "game_ids": [    "GAME003",    "GAME006"  ],  "player_count": 5}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  "http://127.0.0.1:8000/sessions/start" \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "session_id": "SESSION002",  "group_name": "College Legends",  "table_id": "TABLE001",  "game_ids": [    "GAME003",    "GAME006"  ],  "player_count": 5}
```

Request URL

http://127.0.0.1:8000/sessions/start

Server response

Code

Details

200

Response body

```
{  "session_id": "SESSION002",  "group_name": "College Legends",  "table_id": "TABLE001",  "game_ids": [    "GAME003",    "GAME006"  ],  "player_count": 5}
```

Description:

Shows successful creation of a customer gaming session and allocation of a table.

Screenshot 6 – Session Details

Sessions

GET

/sessions/

Get All Sessions

⌵

GET

/sessions/{session_id}

Get Session

⌵

DELETE

/sessions/{session_id}

Delete Session

⌵

POST

/sessions/start

Start Session

⌵

POST

/sessions/end/{session_id}

End Session

⌵

POST

/sessions/{session_id}/add-game

Add Game To Session

⌵

GET

/sessions/details/{session_id}

Get Session Details

⌶

Return full details of a session using an aggregation pipeline.

The response includes:

- The session document itself
- "table_info" -> details of the table used in this session
- "games_info" -> details of all games used in this session

Parameters

Cancel

Name	Description
session_id * required	
string	
(path)	

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/sessions/details/SESSION002' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/sessions/details/SESSION002
```

Server response

Description:

Displays complete session information including allocated table and selected games.

Screenshot 7 – Dashboard Report

The screenshot displays a REST client interface with the following sections:

- Reports**: The main title of the interface.
- Parameters**: A section for defining request parameters, currently showing "No parameters".
- Execute**: A blue button to execute the request.
- Clear**: A button to clear the request.
- Responses**: A section for viewing the response details.
- Curl**: A text area containing the curl command:

```
curl -X 'GET' \
  'http://127.0.0.1:8000/reports/dashboard' \
  -H 'accept: application/json'
```
- Request URL**: A text area showing the URL: `http://127.0.0.1:8000/reports/dashboard`.
- Server response**: A section showing the response status and body.
- Code**: A table with two columns: "Code" and "Details".
- 200**: The response status code.
- Response body**: A text area showing the JSON response:

```
{
  "total_categories": 5,
  "total_games": 10,
  "total_tables": 5,
  "total_sessions": 2,
  "active_sessions": 1,
  "completed_sessions": 1,
  "available_games": 8,
  "games_in_use": 2,
  "available_tables": 4,
  "occupied_tables": 1
}
```

Description:

Displays overall cafe statistics and operational summary.

Screenshot 8 – Most Played Games Report

Reports ^

GET /reports/dashboard Dashboard Report v

GET /reports/session-history Session History v

GET /reports/most-played-games Most Played Games ^

Parameters Cancel

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/reports/most-played-games' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/reports/most-played-games

Server response

Code	Details
200	Response body

```
← → ↺ ⓘ 127.0.0.1:8000/reports/most-played-games
Pretty-print ☒
[
  {
    "game_id": "GAME006",
    "game_name": "Ludo",
    "times_played": 2
  },
  {
    "game_id": "GAME003",
    "game_name": "Risk",
    "times_played": 1
  },
  {
    "game_id": "GAME001",
    "game_name": "Catan",
    "times_played": 1
  },
  {
    "game_id": "GAME002",
    "game_name": "Chess",
    "times_played": 1
  }
]
```

Description:

Displays games ranked according to usage frequency.

Screenshot 9 – Most Used Tables Report

Reports ^

GET /reports/dashboard Dashboard Report v

GET /reports/session-history Session History v

GET /reports/most-played-games Most Played Games v

GET /reports/most-used-tables Most Used Tables ^

Parameters Cancel

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/reports/most-used-tables' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/reports/most-used-tables
```

Server response

Code	Details
200	Successful

```
← → ↺ ⓘ 127.0.0.1:8000/reports/most-used-tables
Pretty-print ☒
[
  {
    "table_id": "TABLE005",
    "table_name": "Castle Table",
    "sessions_hosted": 1
  },
  {
    "table_id": "TABLE001",
    "table_name": "Dragon Table",
    "sessions_hosted": 1
  },
  {
    "table_id": "TABLE002",
    "table_name": "Knight Table",
    "sessions_hosted": 1
  }
]
```

Description:

Displays tables ranked according to number of sessions hosted.

Screenshot 10 – MongoDB Compass Collections

- GAME TABLE-

Welcome

localhost:27017

games

tables

categories

sessions

+

localhost:27017 > boardgame_cafe > games

Documents 10 Aggregations Schema Indexes 1 Validation

1 {}

Generate query

Explain

Reset

Find

Options

ADD DATA

BULK

EXPORT DATA

EXPORT CODE

100 1 - 10 of 10

games

	_id ObjectId	game_id String	name String	category_id String	min_players Int32	max_players Int32	status Str
1	ObjectId('6a30344bf1df7f0...	"GAME001"	"Catan"	"CAT001"	3	4	"Available"  
2	ObjectId('6a303459f1df7f0...	"GAME002"	"Chess"	"CAT001"	2	2	"Available"  
3	ObjectId('6a32eb34f609c5c...	"GAME003"	"Risk"	"CAT001"	2	6	"Available"  
4	ObjectId('6a32eb4bf609c5c...	"GAME004"	"Monopoly"	"CAT004"	2	6	"Available"  
5	ObjectId('6a32eb5bf609c5c...	"GAME005"	"Uno"	"CAT003"	2	10	"Available"  
6	ObjectId('6a32eb76f609c5c...	"GAME006"	"Ludo"	"CAT004"	2	6	"Being Pla"  
7	ObjectId('6a32eb84f609c5c...	"GAME007"	"Scrabble"	"CAT004"	2	4	"Available"  
8	ObjectId('6a32eb8ef609c5c...	"GAME008"	"Sudoku Challenge"	"CAT005"	1	4	"Available"  
9	ObjectId('6a32eb98f609c5c...	"GAME009"	"Ticket to Ride"	"CAT001"	2	5	"Available"  
10	ObjectId('6a33f558aa82a28...	"GAME010"	"Cards Against Humanity"	"CAT002"	4	12	"Available"  

- TABLES TABLE-

Welcome

localhost:27017

games

tables

categories

sessions

+

localhost:27017 > boardgame_cafe > tables

Documents 5 Aggregations Schema Indexes 1 Validation

1 Type a query: { field: 'value' } or Generate query

Explain

Reset

Find

Options

ADD DATA

BULK

EXPORT DATA

EXPORT CODE

25 1 - 5 of 5

tables

	_id ObjectId	table_id String	name String	capacity Int32	status String	
1	ObjectId('6a32e0a3f1df7f0...	"TABLE001"	"Dragon Table"	4	"Available"	 
2	ObjectId('6a32e0bbf1df7f0...	"TABLE002"	"Knight Table"	6	"Available"	 
3	ObjectId('6a32ebddf609c5c...	"TABLE003"	"Wizard Table"	8	"Available"	 
4	ObjectId('6a32ebfef609c5c...	"TABLE004"	"Phoenix Table"	4	"Available"	 
5	ObjectId('6a32ebfcf609c5c...	"TABLE005"	"Castle Table"	10	"Available"	 

- CATEGORY TABLE-

Welcome

localhost:27017

games

tables

categories

sessions

+

localhost:27017 > boardgame_cafe > categories

Documents 5 Aggregations Schema Indexes 1 Validation

1 Type a query: { field: 'value' } or Generate query

Explain

Reset

Find

Options

ADD DATA

BULK

EXPORT DATA

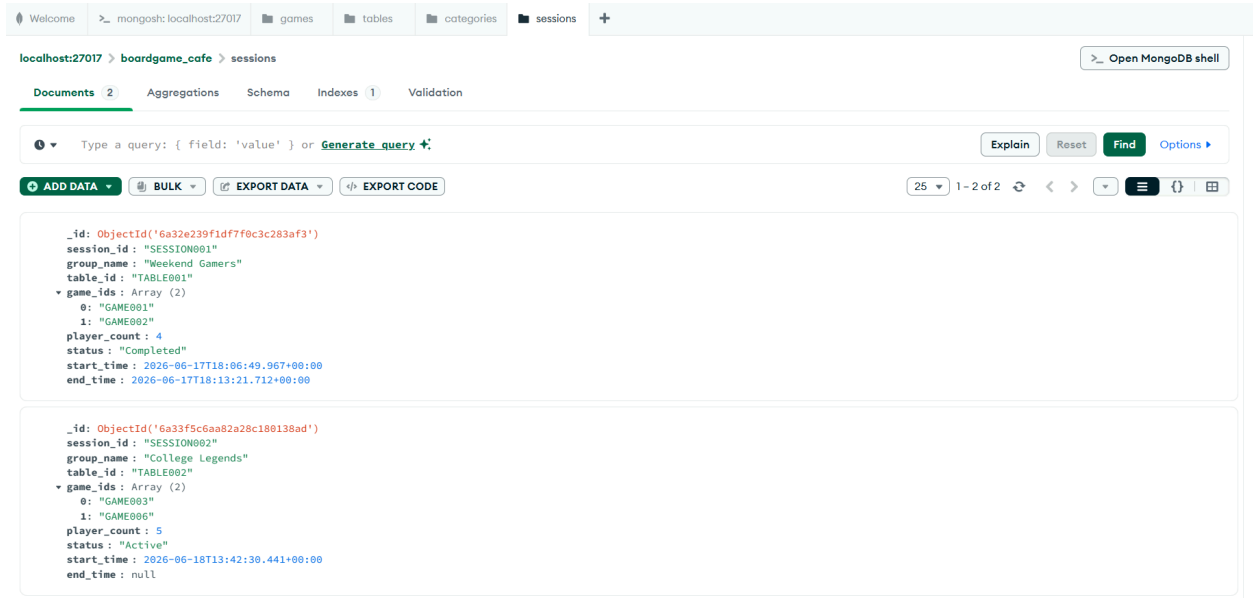
EXPORT CODE

25 1 - 5 of 5

categories

	_id ObjectId	category_id String	name String	description String	
1	ObjectId('6a3033fcf1df7f0...	"CAT002"	"Party"	"Party games"	 
2	ObjectId('6a303432f1df7f0...	"CAT003"	"Card"	"Card based games"	 
3	ObjectId('6a32e9a6f609c5c...	"CAT004"	"Family"	"Family-friendly board ga...	 
4	ObjectId('6a32e9bbf609c5c...	"CAT005"	"Puzzle"	"Logic and puzzle solving...	 
5	ObjectId('6a33f49faa82a28...	"CAT001"	"Strategy"	"Strategic planning and d...	 

- SESSIONS TABLE-



Description:

Displays stored data in MongoDB collections.

13. Summary

The Board Game Cafe Management System successfully demonstrates the integration of FastAPI and MongoDB to build a real-world backend application. The system supports collection relationships, CRUD operations, session management, and reporting functionalities. Additional analytics features provide valuable insights into game popularity and table utilization.

14. Conclusion

The project successfully fulfills all requirements of a MongoDB-based backend application. It demonstrates database design, collection relationships, REST API development, business logic implementation, and data analytics using MongoDB Aggregation Pipelines. The system provides an efficient solution for managing a board game café while showcasing practical backend development skills and database management concepts.